Ejercicio 1

a) Implemente la clase `String` en C++ para representar cadenas de a lo sumo MAX caracteres. Internamente, la clase utilizará `char *` como representación, con uso de memoria dinámica.

Se deben proveer los métodos usuales (constructor por defecto, constructor común, constructor de copia, destructor y sobrecarga del operador de asignación) en los cuales se debe manejar cuidadosamente la memoria dinámica evitando alias, y además las siguientes operaciones:

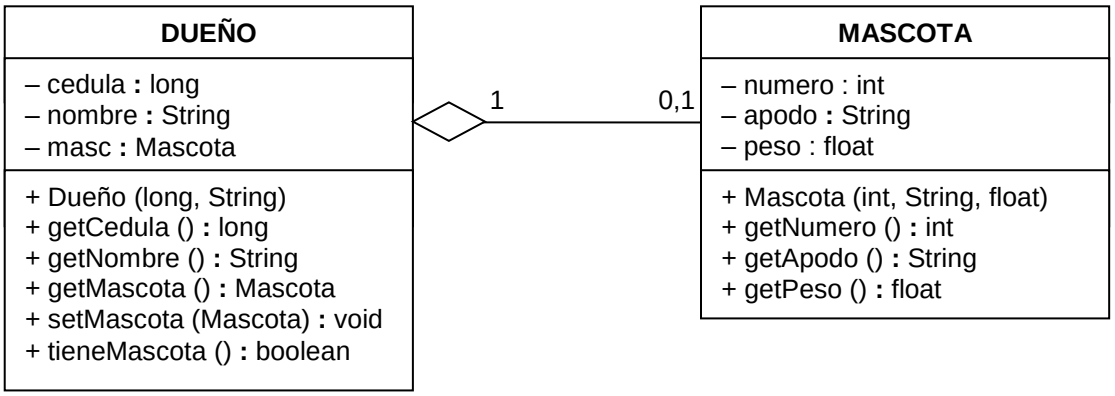
```
==  : String x String -> boolean
<   : String x String -> boolean      (comparación alfabética)
+   : String x String -> String       (concatenación)
```

También se deben programar las operaciones `scan` y `print`, que leen un string de teclado y muestran un string por pantalla respectivamente.

b) Haga un programa de prueba (`main`) que pruebe los métodos de la clase `String`.

Ejercicio 2

Considere el siguiente fragmento de un Diagrama de Clases de Implementación para un certamen de mascotas:



Los dueños son creados originalmente sin mascota y posteriormente se les puede asignar una mediante el método `setMascota`. Se desea garantizar el ***principio de no duplicación de objetos*** para la clase Mascota.

a) Implemente la clase **Mascota** en C++ con los métodos propuestos en el diagrama. Incluya además constructor por defecto y destructor.

b) Implemente la clase **Dueño** en C++ con los métodos propuestos en el diagrama. Incluya además constructor por defecto, constructor de copia, sobrecarga de asignación y destructor. Su implementación debe ser tal que en ningún momento se realicen copias de la mascota. Al destruir un dueño, se debe destruir también su mascota (en caso de tenerla).

c) Haga un programa de prueba (`main`) que pruebe los métodos de ambas clases. Realice pruebas tanto con **objetos** como con **punteros a objetos**.

Este material es de uso exclusivo para los cursos impartidos por Universidad de la Empresa

Ejercicio 3

Dada la siguiente especificación en UML para un ***iterador*** de Dueños del ejercicio anterior:

<< colección >> **ITERADOR**
– iter: LPPF
+ Iterador () + insertar (Dueño) **:** void + hayMasDueños () **:** boolean + proximoDueño () **:** Dueño

Implemente la clase `Iterador` en C++ usando una estructura de ***Lista con puntero al principio y puntero al final*** según la siguiente definición. Luego haga un programa de prueba (`main`) que pruebe los métodos del iterador.

```
private:  struct Nodo {
             Duenio * info;     //iterador de punteros a dueños
             Nodo * sig;
          };
          Nodo * prim;  Nodo * ulti;  Nodo * actual;
```

Notas:

- Incluya solamente constructor ***por defecto*** en la implementación.
- `prim` y `ulti` apuntan al primer y último nodo respectivamente.
- `actual` indica la posición actual del iterador.
- La inserción debe hacerse siempre por el final de la LPPF.

Ejercicio 4

Dada la siguiente especificación en UML para un ***diccionario*** de Mascotas y un ***diccionario*** de Dueños (del ejercicio 2):

<< diccionario >> **MASCOTAS**
– hash: Hash
+ Mascotas () + member (int) **:** boolean + insert (Mascota) **:** void + find (int) **:** Mascota + contarPeso (float) **:** int

<< diccionario >> **DUEÑOS**
– abb: ABB
+ Dueños () + member (long) **:** boolean + insert (Dueño) **:** void + find (long) **:** Dueño + listarDueños (Iterador &) **:** void

- La clave que identifica a cada Mascota es su número de mascota y la clave que identifica a cada Dueño es su cédula.
- Los métodos primitivos **member**, **insert** y **find** tienen el comportamiento usual de las primitivas de diccionario.
- El método específico **contarPeso** de la clase `Mascotas` devuelve la cantidad de mascotas del diccionario que tienen dicho peso.
- El método específico **listarDueños** de la clase `Dueños` devuelve un iterador conteniendo a todos los dueños, ordenado por cédula de menor a mayor.

Este material es de uso exclusivo para los cursos impartidos por Universidad de la Empresa

a) Implemente la clase `Mascotas` en C++ usando una estructura de ***Hash*** según la siguiente definición y haga un programa de prueba (`main`).

```
private:  struct Nodo {
             Mascota * info;   //colección de punteros a mascotas
             Nodo * sig;
          };
          Nodo * hash[B];
```

b) Implemente la clase `Dueños` en C++ usando una estructura de ***Árbol Binario de Búsqueda*** según la siguiente definición y haga un programa de prueba (`main`).

```
private:  struct Nodo {
             Duenio * info;   //colección de punteros a dueños
             Nodo * hizq;
             Nodo * hder;
          };
          Nodo * abb;
```

Notas:

- En ambos casos se pide una implementación de ***punteros a objetos***.
- Incluya solamente constructor ***por defecto*** en ambas implementaciones.
- Además de las operaciones especificadas, implemente también el método ***destructor*** de modo que borre físicamente a todos los elementos de cada colección. Defina una estrategia de borrado tal que cada mascota se elimine **una** sola vez.
- En la parte (a) asuma que la función de dispersión es **h(x) = x % B**.
- Cualquier operación auxiliar que se utilice en forma interna a la clase deberá declararse como ***privada***. Los ***únicos*** métodos visibles desde fuera de la clase deben ser estrictamente los declarados en la especificación en UML.

Ejercicio 5

Dada la siguiente especificación en UML para una clase que representa una ***Fachada*** de acceso a la capa lógica para el sistema del certamen de mascotas:

<< fachada >> **CAPA LOGICA**
– mascotas: Mascotas – dueños: Dueños
+ CapaLogica () + altaDueño (Dueño, tipoError &) **:** void + asignarMascota (long, Mascota, tipoError &) **:** void + listarDueños (Iterador &) **:** void + contarPeso (float) **:** int

- El constructor solamente crea los diccionarios vacíos.
- El método **altaDueño** produce un error si el dueño ya existía
- El método **asignarMascota** da un error si el dueño no existe o si existe pero ya tenía mascota.
- Los métodos **listarDueños** y **contarPeso** invocan a sus homónimos en los diccionarios.

a) Defina un módulo **TipoError** conteniendo un enumerado para los tres tipos de error posibles.

b) Implemente la clase **CapaLogica** en C++ incluyendo únicamente los cinco métodos definidos en la especificación.

c) Haga un programa de prueba (`main`) que pruebe los métodos de la clase **Fachada**. Al realizar las pruebas, asegúrese de trabajar tanto con dueños con mascota como con dueños sin mascota. Trabaje en todo momento con **punteros a objetos**.

Este material es de uso exclusivo para los cursos impartidos por Universidad de la Empresa